

UrbanCool Plan Revision

Every API line is now settled against the live key, not the documentation.
Ten hold. Two do not: Abu Dhabi has no data, and neither does the forecast.



2 BLOCKING FINDINGS	1.80 M CREDITS LEFT OF 2 M	~40 h DATA LAG BEHIND NOW
2 days TO THE 30 AUG DEADLINE		

Two things the key disproved

Failure is silent — `Completed` does not mean you got data

READ THIS ONE TWICE

The single most dangerous behaviour in this API. Everything below is a consequence of it.

A heatmap request for an area or a time the API has no data for is **not** rejected. It returns `200`, it is assigned an `activity_id`, it polls through to `status: "Completed"`, and it hands back an empty `FeatureCollection`. Nothing anywhere in the response says “no data”. The only tell is the shape of `stats_data`:

```
// data present
"stats_data": { "temperature_stats": {...}, "overall_temperature_distribution": [...], ...
}

// NO DATA — same 200, same "Completed"
"stats_data": { "activity_id": "...", "n_cells": 0 }
"map_data": { "type": "FeatureCollection", "features": [] }
```

Built naively, the dashboard shows a blank map with a green last-updated timestamp and no error in any log.

Assert on `map_data.features.length > 0` before caching or rendering anything — treat an empty result as a failure, not a success. This cost the first verification run a false pass on every check; the harness now fails on zero tiles.

Abu Dhabi has no data — the AOI must move

CONFIRMED LIVE

Tested directly, at a date and hour proven to return data elsewhere in the same run:

```
masdar    2024-07-15 14:00Z    tiles=0    EMPTY
phoenix    2024-07-15 14:00Z    tiles=74    mean=39.7 degC
```

The documentation was right after all — coverage is United States only, and the undocumented `Hackathon` plan tier is no exception. The submission being accepted proved nothing, because of the silent-empty behaviour above.

Move the AOI to downtown Phoenix. The swap is surgical: city name, polygon, and regulatory hook. Four zones, thresholds, contracts, cloud, and the whole computer-vision workstream are untouched. The polygon in `config/aoi.json` returns 74 tiles at 100 m and 198 at 60 m.

- **The heat is real** — 42.9 °C mean across the AOI on 26 Aug.
- **The regulatory hook survives** — Arizona ADOSH heat rules and Phoenix's contractor heat-safety ordinance replace the UAE midday break.
- **The cool point improves** — Phoenix publishes an official cooling-centre dataset.
- **Mapillary coverage is denser** in Phoenix than Masdar.

The forecast returns nothing — “predictive” has to be re-grounded

CONFIRMED LIVE

The 12-hour horizon is documented, and future timestamps are accepted at submit. They come back empty. So does everything in the last ~40 hours:

```
now - 48h (08-26 05:00Z)  tiles=74  mean=32.9 degC
now - 36h (08-26 17:00Z)  tiles=0    EMPTY
now - 24h                  tiles=0    EMPTY
now - 3h                   tiles=0    EMPTY
now + 3h / +6h / +11h     tiles=0    EMPTY // the "forecast"
```

The dataset ends at **26 Aug 13:00Z** and lags real time by roughly 40 hours. There is no live reading and no forward projection to be had. Two consequences, neither fatal:

- **The demo runs on a fixed date.** Pin it to 26 Aug 2026 and say so plainly on screen. The plan already committed to fixtures and to labelling stale data honestly — this is that policy, applied harder.
- **“When it peaks” comes from `time_of_measure`, not a forecast.** It returns the peak hour per tile over a chosen day and it works: `{"tile_id":0,"value":15}` — peak at 15:00 UTC. That is the panel line the demo story needs, sourced from real data rather than a projection that does not exist.

Describe the system as *heat-risk assessment with within-day peak prediction*. Claiming a live 12-hour forecast in front of judges who know this API is a risk not worth taking.

DECISIONS 01 AND 02

What is confirmed and what moves

01 · Environmental & Health Monitoring — confirmed

AGREED

It is the right call from the list. Heat-risk alerting is the product, not a by-product, and it is the only option that needs temperature, forecast and context layers together. Urban Design & Public Space Optimization as a secondary framing is fine — the hackathon rules say “*select one challenge, or combine multiple.*”

One thing to check on the submission form

VERIFY

The seven tracks published on the hackathon page are named differently from the seven options in hand: Resilient Cities & Infrastructure · Future Buildings & Energy · Industrial & Enterprise · Government & Environment · Model Designing · Agentic AI · Data Analysis & Correlation.

Neither list contains the other's names, so one of them is not the submission taxonomy. Open the submission form and check which set it offers. If it is the published set, UrbanCool maps to **Government & Environment** first — outdoor-worker safety against a regulatory rule — with **Resilient Cities & Infrastructure** second. Five minutes now; a mis-filed submission is not recoverable on 30 August.

02 · City — Abu Dhabi is not available; move to Phoenix

CHANGED

Three published sources agree that coverage is United States only, so this is a constraint to design around rather than a question to test. Redraw the polygon on downtown Phoenix and re-pick the four zones against it; a candidate 1.1 × 0.9 km polygon is already in `config/aoi.json`.

Everything downstream — zone logic, thresholds, contracts, cloud, computer vision — is city-independent and was never blocked by this.

SECTION 03, REVISED

All twelve API lines, answered

Everything below was exercised against the live key on 28 Aug. Ten checks pass, two fail. Re-runnable at any time with `npm run verify`, which writes the same table to `out/verification-report.md`.

PLAN LINE	VERDICT	WHAT THE LIVE KEY RETURNED
Authentication	● holds	<code>api-key</code> request header against <code>https://api.fortyguard.com</code> . A call without it returns a clean <code>401</code> with <code>details.message</code> . Errors are shaped <code>{error, status_code, details:{message}}</code> — Essam can key the fallback off that.
Endpoints	● answered	Six, as expected: <code>POST /v1/heatmap</code> , <code>POST /v1/env_params</code> , <code>POST /v1/satellite</code> , <code>POST /v1/streetview</code> , <code>POST /v1/heat_intelligence</code> , <code>GET /v1/status/{id}</code> . Satellite, Street View and Heat Intelligence are Premium only — not on our plan.
Async pattern	● holds	Submit returns <code>data.activity_id</code> ; poll <code>GET /v1/status/{id}</code> until <code>data.status</code> is <code>Completed</code> or <code>Failed</code> . Both are terminal.
Request schema	● correction	<code>polygon_aoi</code> must be a GeoJSON FeatureCollection wrapping a Feature wrapping the Polygon — not the bare Polygon in section 02. See the correction below.

PLAN LINE	VERDICT	WHAT THE LIVE KEY RETURNED
Max AOI & granularity	● measured	Our ~1 km ² AOI yields 74 tiles at 100 m and 198 at 60 m ; a 5.5 × 4.5 km box yielded 2,288. Nowhere near the 10 mi ² ceiling. Use 60 m — four zones separate far better across 198 tiles than 74.
Credits	● drop the rule	Plan tier is Hackathon — undocumented, not in the plan table. 2,000,000 credits , key valid to 1 Oct 2026 . Measured cost: 4,220 per heatmap call, 2,900 per env_params call — about 470 heatmaps' worth. Verification spent 200 k (10%); 1.80 M remain. The 60-calls-a-week rule serves no purpose.
Beyond air temperature	● better, with a catch	All 15 parameters returned, not the documented 3-parameter cap — humidity, heat index, wet bulb, AQI, solar. Open-Meteo is not needed . The catch: <code>env_params</code> takes <code>temperature</code> as an <i>input</i> , and derives heat index from whatever you pass. Feed it the real tile temperature or the output is confidently wrong.
Historical range	● generous	Back to 2019-01-01 , far beyond the seven days we asked for. <code>filter_type: 4</code> (range of days) works despite the Known Limitations page omitting it — a week of history is one call , not seven.
Forecast horizon	● no data	Accepted at submit, empty on return , at +3 h, +6 h and +11 h alike. The dataset ends 26 Aug 13:00Z, ~40 h behind now. There is no live value and no forward projection. Use <code>time_of_measure</code> for peak hour instead — see the finding above.
Analysis layer	● two shapes	“Snapshot” is <code>tcm</code> (the default). The two layer families return different tile properties — parse accordingly: <code>tcm</code> → <code>{tile_id, average_temperature, min_temperature, max_temperature}</code> <code>exceedance / time_of_measure / persistence</code> → <code>{tile_id, value}</code> in hours, with <code>stats_data</code> carrying <code>analytic_type, units, min, max, mean</code> .
Rate limits & latency	● measured	Twelve concurrent status calls, no 429 . Jobs complete in 2–32 s (often on the first poll). A 5 s poll with a 2-minute ceiling is comfortably safe; the 30-minute cache refresh is far more conservative than needed.
Failure behaviour	● holds	<code>400</code> bad input · <code>401</code> missing key · <code>429</code> rate limit · <code>500</code> server. Serving the last cached value with a visible timestamp stays the right fallback.

SECTION 05, CORRECTIONS

Four things to fix before Essam builds against the contract

The contracts are frozen, which is right — but four details in them will not survive contact with the API or with the demo. Cheaper to fix now than at integration.

1 · The AOI needs a FeatureCollection wrapper

WILL 400

The polygon in section 02 is a bare Polygon. The API requires the full wrapper:

```
"polygon_aoi": {
  "type": "FeatureCollection",
  "features": [{
    "type": "Feature",
    "properties": {},
    "geometry": {
      "type": "Polygon",
      "coordinates": [[ /* lon, lat - closed ring */ ]]
    }
  }]
}
```

Handled by `featureCollection()` in `src/fg/client.js` — pass a bare ring, get a valid payload.

2 · centroid and coordinates disagree on axis order

SILENT BUG

The AOI ring is longitude-first (correct GeoJSON). The dashboard contract's `"centroid": [24.4297, 54.6168]` is latitude-first. Both are “right” by some convention, which is exactly why this kind of mismatch survives review and then puts a zone in the wrong hemisphere. Pick one and write it into the field name — `centroid_lat_lon` ends the argument permanently.

3 · The worked example does not match the contract, and proves nothing either way

TWO PROBLEMS

Section 04 computes $51.4 + 2 + 3 + 0 - 0 = 56.4$. The section 05 contract for the same zone says `"risk_score": 53.4` — which is $51.4 + 2$, the worker penalty dropped, even though `drivers` lists `"outdoor workers present"`. One of the two is wrong.

The deeper problem: a heat index of 51.4°C is *already* Extreme before a single contextual point is added. The demo's centrepiece zone therefore demonstrates nothing about the contextual engine — the clock alone would produce the same answer. Pick a worked example where context does the work:

```
// Z-02, 13:40 — context moves the level
heat_index      49.0  -> Danger on its own
+ peak_hour      +2.0
+ worker         +3.0
= risk_score     54.0  -> Extreme
```

Now the zone is Extreme *because* workers are present at peak hour. That is the story the demo is built to tell.

4 · The timestamps are in the wrong timezone for the story

DEMO-BREAKING

The demo is a foreman at **13:40 local**, at the hottest part of the day. The contract stamps that as `"2026-08-28T13:40:00Z"` — but in Phoenix (UTC-7) 13:40 local is `20:40Z`, and `13:40Z` is 06:40 in the morning. The `"forecast_peak_time": "...T15:00:00Z"` lands at 08:00 local, which is not a heat peak anywhere.

Store UTC, render local, and put the offset in the payload so the dashboard cannot get it wrong.

`/v1/env_params` hands you `metadata.timezone` and `metadata.timezone_offset_hours` — use them rather than hardcoding. Note that `time_of_measure` returns its peak hour in **UTC**, so it needs the same treatment.

FortyGuard computes the heat index for you

Stage one gets shorter, and better sourced

CONFIRMED

Stage one plans to compute the heat index from temperature and humidity using the NWS regression.

`/v1/env_params` returns `heat_index_celsius` directly — so the value comes from the same API as the temperature, with no second source to explain and no regression to defend.

Our plan allows three parameters per request. Take `heat_index_celsius`, `relative_humidity_percent` and `wet_bulb_temperature_celsius`. The third is worth the slot: wet-bulb temperature is the measure occupational heat-stress standards are actually written against, and having it on the evidence slide is a stronger answer to “why these thresholds” than a citation alone.

The five bands themselves are sound — `27 / 32 / 39 / 51 °C` are the NWS heat-index categories converted from `80 / 90 / 103 / 125 °F`. Keep them, and cite the conversion.

NOBODY IS BLOCKED

What each person does next

SHAHAD

Computer vision — unblocked, start now

- The contract in section 05 is unaffected by the city decision. Build against it.
- Satellite and Street View segmentation are **Premium only** — FortyGuard will not supply shade percentage. Decision 07 was right: our own CV is load-bearing, not a nice-to-have.
- The AOI is settled — pull Mapillary for the downtown Phoenix bounding box, not Masdar.
- Pin the fixed demo stills to **26 Aug 2026** so the vision output and the temperature data describe the same day.

ESSAM · BACKEND

Poll worker — unblocked, start now

- Submit-and-poll is confirmed. `src/fg/client.js` already implements it against the real shapes.
- Cache on `{aoi, timestamp, analytic_type, granularity}`.
- Parse `data.result.map_data` (GeoJSON tiles) and `data.result.stats_data` (min/mean/max/ σ plus distributions).
- Key the fallback off `details.message`.

EXPERIENCE

Recommendation copy — one dependency

- The recommendation codes are city-independent. Write them now.
- `r_stop_work` is the exception — its wording cites a specific rule, so it waits on decision 02.
- If the city moves, `r_nearest_cool_point` gets *better*: Phoenix publishes an official cooling-centre dataset.

SETTLED, AND RE-RUNNABLE

The verification run

Already run against the live key. Ten checks pass; the two failures are the Masdar coverage result and the empty forecast, both recorded deliberately rather than papered over.

```
npm run verify           # ten checks, ~30 k credits
npm run verify:usage     # plan + balance only, spends nothing
node scripts/coverage-probe.js # is a given AOI covered at all?
```

`coverage-probe.js` is the one to keep. Any new area of interest goes through it first — it submits, polls, and reports the tile count, so an uncovered AOI is caught in a minute instead of surfacing as a blank map on stage. Given how the API reports absence, that check belongs in front of every location decision from here.

UrbanCool · coordination plan v3.2 Verified against the live key, 28 Aug 2026

Submission 30 Aug, 23:59 GST 10 of 12 checks pass · re-run with `npm run verify`